

Product Requirements Document (PRD)

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Status:** Draft for Engineering Review **Owner:** Product Management **Based On:** Product Vision v1.0

1. Purpose of This Document

This PRD translates the Product Vision into concrete, buildable requirements. It defines **what** the product must do (features, user stories, acceptance criteria), **how well** it must do it (non-functional requirements), and **in what order** it gets built (prioritization and release plan). It is the primary reference for Engineering, Design, and QA during implementation.

2. Product Summary

SentinelCut lets a user upload an audio/video file, automatically detects profane/offensive words in English, Hindi, Hinglish, and mixed-language speech, and produces a censored output (beep/mute/custom sound) with a human-editable review timeline before final export.

3. User Personas (Reference)

Persona	Primary Need
Independent Creator (Rohan)	Fast, self-serve censorship for YouTube/podcast uploads
Post-Production Editor (Priya)	Bulk, reliable censorship with fine-grained manual override
L&D/EdTech Manager (Ankit)	Clean up recorded lecture/webinar audio before distribution
Guest/Evaluator	Try the product with zero signup friction (demo purposes)

4. User Stories & Functional Requirements

Each feature area lists **user stories**, **functional requirements (FR)**, and **acceptance criteria (AC)**.

4.1 Authentication & Account

User Stories

- As a new user, I want to sign up with email/password or Google so I can save my work.
- As a returning user, I want to log in and see my past files.

- As a first-time visitor, I want to try the product as a guest without creating an account.
- As a user who forgot my password, I want to reset it via email.

Functional Requirements

ID	Requirement
FR-1.1	System shall support email/password signup and login
FR-1.2	System shall support Google OAuth login
FR-1.3	System shall support "Forgot Password" via emailed reset link
FR-1.4	System shall support Guest Mode with session-limited access (no persistent history)
FR-1.5	System shall enforce password strength rules (min 8 chars, 1 number, 1 special char)

Acceptance Criteria

- AC-1: User can complete signup → verify email → login in under 2 minutes.
- AC-2: Guest sessions expire after a configurable time (e.g., 24 hours) and clearly warn the user their files are temporary.
- AC-3: Password reset link expires after 30 minutes for security.

4.2 Dashboard

User Stories

- As a logged-in user, I want to see my recent files, processing status, and remaining credits at a glance.

Functional Requirements

ID	Requirement
FR-2.1	Dashboard shall display Recent Files (last N uploads with status)
FR-2.2	Dashboard shall display History with search/filter by date, status, filename
FR-2.3	Dashboard shall display Credits/Usage remaining
FR-2.4	Dashboard shall display Storage Usage (used vs. limit)
FR-2.5	Dashboard shall display live Processing Queue status for in-progress jobs

Acceptance Criteria

- AC-1: Dashboard loads within 2 seconds for a user with up to 100 historical files.
- AC-2: Processing status updates in near real-time (polling or websocket, ≤5s latency).

4.3 Upload

User Stories

- As a user, I want to drag-and-drop or browse to upload my audio/video file.
- As a user, I want clear feedback if my file is too large or an unsupported format.

Functional Requirements

ID	Requirement
FR-3.1	System shall accept video formats: MP4, MOV, MKV, AVI
FR-3.2	System shall accept audio formats: MP3, WAV, AAC, FLAC, M4A
FR-3.3	System shall support drag-and-drop and click-to-browse upload
FR-3.4	System shall enforce a configurable max file size (e.g., 500MB for MVP, tunable per plan)
FR-3.5	System shall show upload progress (%) with the ability to cancel
FR-3.6	System shall validate file integrity/format server-side before queuing for processing

Acceptance Criteria

- AC-1: Unsupported file types are rejected with a clear, specific error message before upload starts.
- AC-2: Uploads over the size limit are blocked client-side with an explanatory message.
- AC-3: A failed/interrupted upload can be retried without re-selecting the file (resumable upload preferred, not mandatory for MVP).

4.4 Detection Mode Selection

User Stories

- As a user, I want to choose automatic detection using a built-in profanity database.
- As a user, I want to define my own custom word list.
- As a user, I want to combine both automatic and custom detection.

Functional Requirements

ID	Requirement
FR-4.1	System shall offer three detection modes: Automatic, Custom, Combined
FR-4.2	Custom mode shall accept a comma/line-separated list of words/phrases
FR-4.3	Combined mode shall merge automatic-database hits with custom-list hits, de-duplicated
FR-4.4	System shall allow saving a custom word list as a reusable preset per user

Acceptance Criteria

- AC-1: Custom words are matched case-insensitively and against common spelling variants.
- AC-2: A user can save, edit, and delete custom word-list presets.

4.5 Language Detection & Speech Recognition

User Stories

- As a user speaking Hinglish, I want the system to correctly transcribe and detect profanity without me specifying the language manually.

Functional Requirements

ID	Requirement
FR-5.1	System shall auto-detect spoken language(s): English, Hindi, Hinglish, mixed
FR-5.2	System shall select the appropriate ASR model/config based on detected language
FR-5.3	System shall handle code-switching within a single file (e.g., sentence starts in English, ends in Hindi)
FR-5.4	System shall allow manual language override if auto-detection is incorrect

Acceptance Criteria

- AC-1: Language auto-detection accuracy $\geq 85\%$ on a labeled test set (English/Hindi/Hinglish).

- AC-2: User can override detected language from a dropdown, triggering re-transcription.
-

4.6 Profanity Detection

User Stories

- As a user, I want the system to catch not just exact profane words but also slang, misspellings, and disguised variants (e.g., "fuuuuck," "m@darchod," "bc").

Functional Requirements

ID	Requirement
FR-6.1	System shall detect profanity via a hybrid approach (dictionary + normalization + context model — see AI Pipeline doc)
FR-6.2	System shall normalize repeated letters (e.g., "fuuuuck" → "fuck") before matching
FR-6.3	System shall detect common abbreviations/slang (bc, mc, bkl, etc.) via curated slang dictionary
FR-6.4	System shall detect leetspeak/symbol substitutions (e.g., "m@darchod") via normalization rules
FR-6.5	System shall assign a confidence score to each detection
FR-6.6	System shall allow filtering detections by confidence threshold in the UI

Acceptance Criteria

- AC-1: Detection recall $\geq 85\%$, precision $\geq 80\%$ on the internal test dataset (per Product Vision success metrics).
 - AC-2: Each detected instance displays word, timestamp, confidence score, and detection source (dictionary/custom/model).
-

4.7 Timeline Editor (Review & Edit)

User Stories

- As a user, I want to see a transcript with detected words highlighted so I can review before finalizing.
- As a user, I want to add a missed detection or remove a false positive.

- As a user, I want to change the replacement type per detection (beep vs. mute vs. custom sound).

Functional Requirements

ID	Requirement
FR-7.1	System shall display full transcript with timestamps aligned to audio/video playback
FR-7.2	System shall visually highlight all detected words/phrases in the transcript
FR-7.3	User shall be able to delete a detection (mark as not profane)
FR-7.4	User shall be able to add a new detection manually by selecting transcript text or a timestamp range
FR-7.5	User shall be able to change replacement mode per individual detection
FR-7.6	User shall be able to adjust the start/end timestamp of a detection (fine-tuning)
FR-7.7	System shall provide inline audio/video playback synced to the transcript cursor
FR-7.8	System shall support "Regenerate Output" after edits without requiring a full re-upload

Acceptance Criteria

- AC-1: Edits made in the timeline are reflected in a live preview before final export.
- AC-2: Regeneration after edits only reprocesses the affected audio segments where feasible (not a full re-transcription), to reduce processing time.
- AC-3: Undo/redo supported for at least the last 10 edit actions.

4.8 Replacement / Censorship Modes

Functional Requirements

ID	Requirement
FR-8.1	System shall support Beep replacement (configurable tone/frequency)
FR-8.2	System shall support Mute (silence) replacement
FR-8.3	System shall support Custom uploaded audio sound as replacement
FR-8.4	System shall reserve extension points for future AI voice-word replacement and voice cloning (not built in MVP)

Acceptance Criteria

- AC-1: Replacement audio duration matches or gracefully fits the original detected word's duration without causing audible desync.

4.9 Export

Functional Requirements

ID	Requirement
FR-9.1	System shall export censored video (matching original container/format where feasible)
FR-9.2	System shall export censored audio-only file
FR-9.3	System shall export subtitle file (.srt) with censored word markers
FR-9.4	System shall export plain transcript (.txt)
FR-9.5	System shall export JSON metadata (detections, timestamps, confidence, actions taken)

Acceptance Criteria

- AC-1: Exported video preserves original resolution, frame rate, and metadata (per Product Vision constraints).
- AC-2: Audio/video sync drift in final export < 100ms (per success metrics).

4.10 Admin Panel (MVP-lite)

Functional Requirements

ID	Requirement
FR-10.1	Admin shall be able to view/manage registered users
FR-10.2	Admin shall be able to manage the global profanity database (add/remove/edit entries)
FR-10.3	Admin shall be able to view uploaded files and processing logs
FR-10.4	Admin shall be able to view basic analytics (uploads/day, avg. processing time, detection stats)
FR-10.5	Admin shall be able to view user feedback submissions

Acceptance Criteria

- AC-1: Admin actions (e.g., editing the profanity database) are logged with admin ID and timestamp.

5. Non-Functional Requirements (NFR)

Category	Requirement
Performance	Processing time $\leq 2 \times$ real-time duration for standard-definition files (per Product Vision)
Scalability	Backend processing shall run via a queue system to support concurrent jobs without blocking the API
Availability	Target 99% uptime for the web application (academic/demo-scale target, not enterprise SLA)
Security	All traffic over HTTPS; JWT-based session auth; input validation on all uploads; virus/malware scanning on uploaded files
Data Privacy	Uploaded files and transcripts are private to the uploading user by default; guest files auto-deleted after expiry
Usability	A first-time user should be able to complete upload $\rightarrow$ detect $\rightarrow$ export without external instructions (per Product Vision task completion metric)
Accessibility	UI shall meet baseline WCAG 2.1 AA contrast/keyboard-navigation guidelines where feasible
Compatibility	Web app shall be responsive across desktop and tablet screen sizes at minimum
Maintainability	Codebase shall follow modular/repository-pattern architecture (per Coding Standards doc) to allow swapping ASR/NLP models
Observability	System shall log processing pipeline stages and errors for debugging and admin visibility

6. User Journeys

6.1 Primary Journey — Registered User, Automatic Detection

1. User logs in $\rightarrow$ lands on Dashboard.
2. Clicks "New Upload" $\rightarrow$ drags in an MP4 file.
3. Selects Detection Mode: Automatic.
4. Selects Output Mode: Beep.
5. Clicks "Start Processing" $\rightarrow$ sees job enter Processing Queue.
6. Receives notification / sees status change to "Ready for Review."

7. Opens Timeline Editor → reviews highlighted detections, removes 1 false positive, adds 1 missed word.
8. Clicks "Regenerate."
9. Previews final output inline.
10. Downloads censored video + transcript.

6.2 Secondary Journey — Guest User, Custom Word List

1. Visitor lands on landing page → clicks "Try as Guest."
2. Uploads a short audio clip.
3. Selects Detection Mode: Custom → enters 3 custom words.
4. Selects Output Mode: Mute.
5. Reviews detections (limited timeline features for guest, e.g., no saved presets).
6. Downloads output before session expires; prompted to sign up to save history.

6.3 Edge Journey — Failed Detection Correction

1. User uploads a file with heavy background noise.
 2. ASR transcription has multiple errors; several profane words are missed.
 3. User manually adds detections by selecting text/timestamps in the editor.
 4. User regenerates and verifies via inline preview before export.
-

7. Edge Cases

Edge Case	Expected Behavior
File with no speech (music only)	System completes transcription with empty/near-empty transcript; no false detections; user notified "No speech detected"
Extremely long file (e.g., 3-hour lecture)	System should chunk processing and show incremental progress; may require async/background job with email notification on completion
Overlapping speakers	Detection still attempted per audio segment; UI should indicate lower confidence where overlap is detected (future: speaker diarization)
Profane word at the very start/end of a clip (partial word cut off)	System should not crash; boundary detection padded by a safety margin (e.g., 100–150ms)
User uploads a corrupted/unsupported codec file	Reject with clear error before entering the processing queue
Same word appears both as profanity and normal word in different context (context-dependent)	Confidence score reflects context-model judgment; user can override via timeline editor
User edits timeline, closes browser mid-edit	Auto-save draft state periodically so work isn't lost
Guest mode session expires before download	Clear warning shown before expiry with a "sign up to save" prompt
Duplicate/near-duplicate custom words entered	De-duplicated automatically, case-insensitive
Non-Latin script custom word input (Devanagari)	Supported; matching pipeline must handle Unicode normalization

8. Assumptions & Constraints

(Inherited from Product Vision; restated here in PRD context)

- MVP targets a reasonable accuracy bar ($\geq 85\%$ recall), not perfection — the human-editable timeline is the safety net.

- Free-tier/open-source infrastructure is used for the academic build; architecture should still be "production-shaped" for future scaling.
 - No proprietary large-scale Hindi/Hinglish profanity dataset exists; detection quality depends on curated dictionaries + open datasets + normalization rules, which is an accepted MVP limitation.
 - Real-time/live censorship, AI voice cloning, and third-party platform integrations are explicitly out of scope for this PRD (see Future Scope in Product Vision).
-

9. Feature Prioritization (MoSCoW)

Must Have (MVP — Release 1)

- Signup/Login (email + Google), Guest Mode
- Upload (drag-and-drop, format/size validation)
- Automatic Detection Mode (dictionary + normalization)
- English + Hindi + Hinglish language auto-detection
- Beep and Mute replacement modes
- Timeline Editor: view transcript, highlight detections, delete/add detection, regenerate
- Export: censored video/audio, transcript (.txt), JSON metadata
- Basic Dashboard: recent files, processing status

Should Have (Release 2)

- Custom Detection Mode + Combined Mode
- Custom word-list presets (save/reuse)
- Custom Sound replacement mode
- Subtitle (.srt) export
- History with search/filter
- Confidence-score filtering in timeline editor
- Basic Admin Panel (user + profanity DB management)

Could Have (Release 3)

- Storage usage & credits system UI
- Undo/redo in timeline editor
- Analytics dashboard for admin
- Forgot password flow polish, rate limiting, virus scan integration
- Dark mode / light mode toggle

Won't Have (This Release — Documented as Future Scope)

- Real-time/live-stream censorship
- AI voice cloning / custom spoken word replacement

- Mobile native apps, browser extensions
- Third-party integrations (Zoom, Discord, OBS, YouTube)
- Multi-tenant enterprise billing

10. Release Planning

Release	Theme	Key Deliverables	Target Milestone
R1 — MVP	Core censorship pipeline works end-to-end	Auth, Upload, Automatic Detection, Beep/Mute, Timeline Editor (basic), Export (video/audio/transcript/JSON)	Mid-semester demo
R2 — Usability & Control	User control and customization	Custom/Combined detection, presets, custom sound, subtitles, history/search	Pre-final demo
R3 — Polish & Admin	Production-shaped polish	Admin panel, analytics, credits/storage UI, UX polish (dark mode, undo/redo)	Final submission/demo
R4+ — Future Scope	Beyond academic scope	Real-time censorship, integrations, voice cloning, mobile app	Post-graduation roadmap

11. Open Questions for Engineering (to resolve in SRS/Architecture phase)

- Exact ASR model selection and its offline/CPU feasibility (addressed in Tech Stack Justification doc).
 - Whether "Regenerate Output" reprocesses only edited segments or the full file for MVP simplicity vs. R2 optimization.
 - Storage retention policy specifics (how long are registered-user files retained by default).
-

12. Next Document

Document 3: Software Requirements Specification (SRS) — formalizing functional and non-functional requirements into IEEE-style specification format, including detailed interface requirements and system constraints.

End of Product Requirements Document